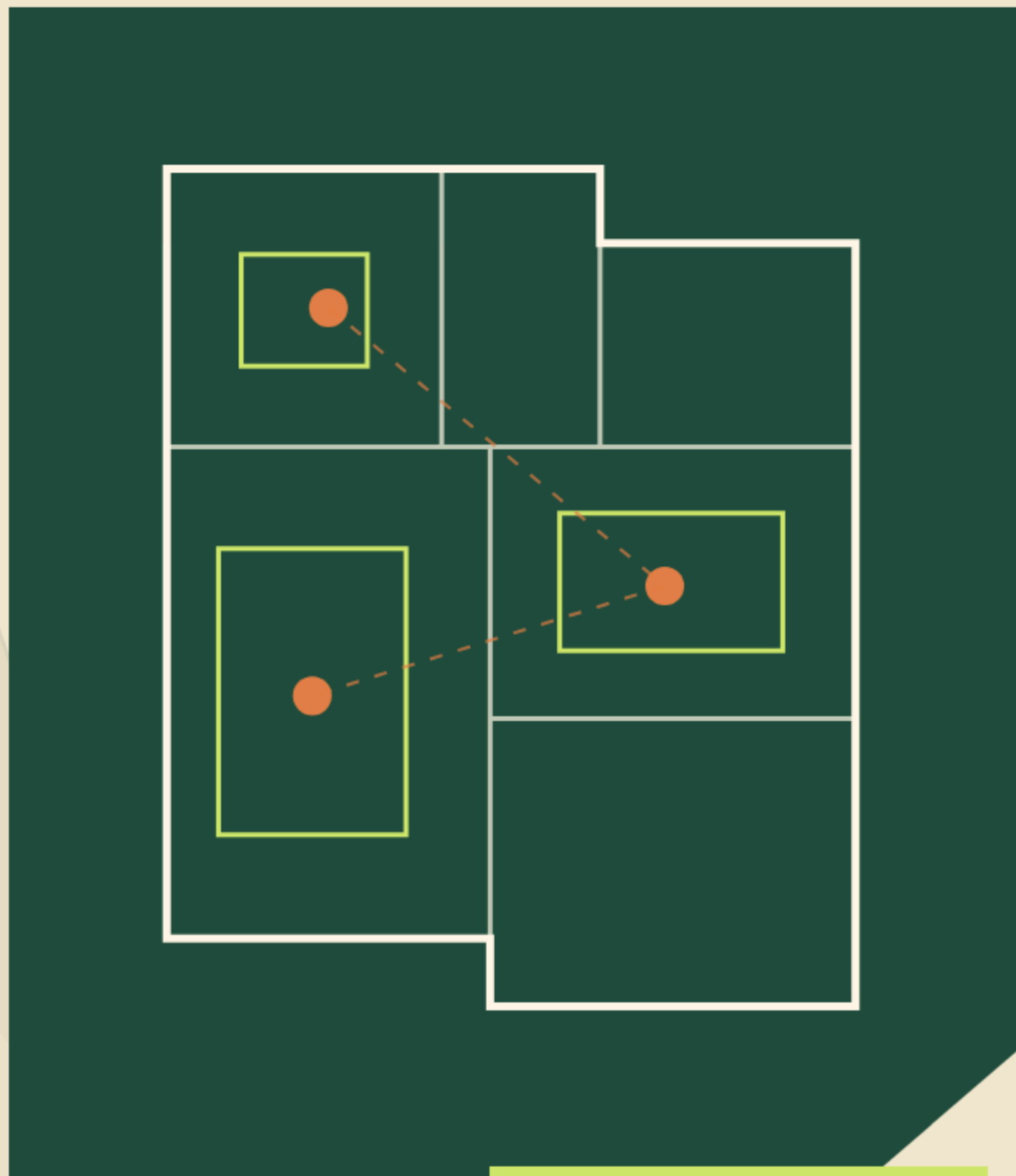


● BÁO CÁO ĐỒ ÁN NT532

FlatMate Comfort

FlatMate mô phỏng một căn hộ nhận biết ngữ cảnh, hiểu yêu cầu tiếng Việt, kiểm tra lệnh trước khi thực hiện và học từ những lần người dùng điều chỉnh.



6,73 × 7,81 m
10 kịch bản tái lập

3D + SSE

CẬP NHẬT THỜI GIAN THỰC

Guardrail

KIỂM TRA TRƯỚC KHI ÁP DỤNG

Bộ nhớ

HỌC SAU 3 LẦN CHỈNH GIỐNG NHAU

Khang Lê · NT532

UIT – VNU-HCM · 11/08/2026

Nhà thông minh hiểu lệnh, nhưng chưa chắc hiểu thế nào là “vừa phải”.

“Phòng rất nóng, làm mát vừa phải.”

“Tôi chuẩn bị làm việc.”

“Không khí hơi ngột ngạt.”



Hành động = $f(\text{yêu cầu, cảm biến, thiết bị, ngữ cảnh, sở thích, lịch sử})$

Mục tiêu là khép kín vòng lặp cảm nhận → xử lý → điều khiển → theo dõi.

01

Thu nhận

Đọc nhiệt độ, độ ẩm, CO₂, PM2.5, ánh sáng, tiếng ồn và hiện diện.

02

Xử lý

Xác định ngưỡng cảnh, tìm sở thích phù hợp, gọi mô hình và ghi nhật ký xử lý.

03

Điều khiển

Backend kiểm tra toàn bộ lệnh trước khi cập nhật thiết bị mô phỏng.

04

Theo dõi

Trạng thái mới được gửi qua SSE đến mô hình 3D, dashboard và lịch sử.

Toàn bộ căn hộ trong một màn hình.

Mô hình căn hộ 3D

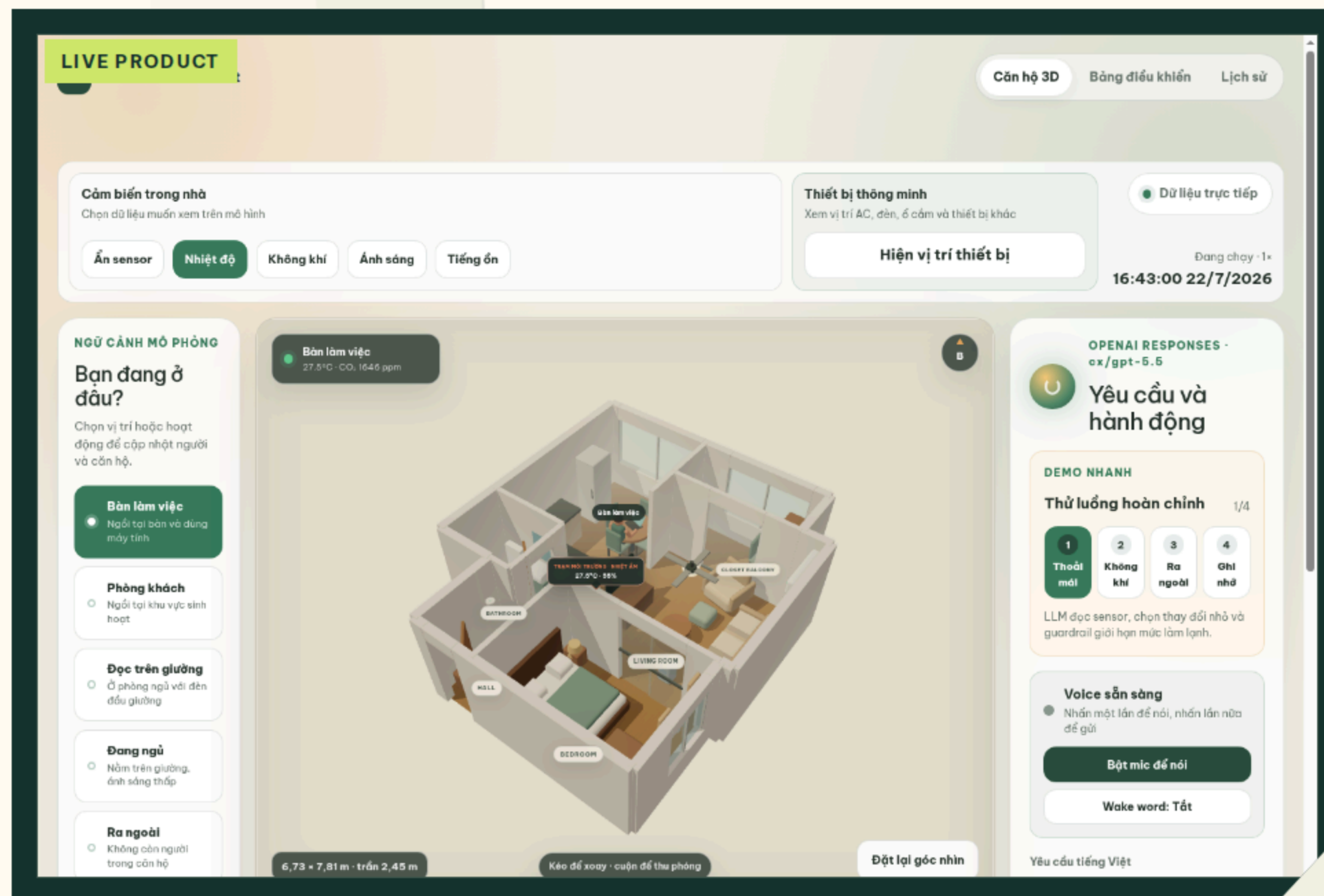
Không gian, người dùng, cảm biến và thiết bị nằm trong cùng một mô hình.

Ngữ cảnh theo thời gian thực

Người dùng có thể chọn làm việc, thư giãn, đọc trên giường, ngủ hoặc ra ngoài.

Theo dõi từng bước xử lý

Mỗi yêu cầu cho biết dữ liệu đầu vào, sở thích đã dùng, công cụ LLM đã chọn, kết quả kiểm tra và trạng thái cuối.



Mọi lớp đều đọc trạng thái hiện tại của căn hộ từ **RoomSnapshot**.

01

Giao diện & giọng nói

Next.js 16 + React 19
Nhập văn bản / giọng nói
3D · dashboard · lịch sử

02

API trợ lý

FastAPI + Pydantic
Xác định ngữ cảnh và sở thích
Luồng gọi công cụ có cấu trúc

03

Kiểm tra & mô phỏng

Kiểm tra toàn bộ lệnh trong scene
Dữ liệu cảm biến theo seed
Mô phỏng tác động thiết bị

04

Dữ liệu & sự kiện

Lịch sử / sở thích trong SQLite
Trạng thái + nhật ký qua SSE
Dữ liệu CSV / JSON

RoomSnapshot

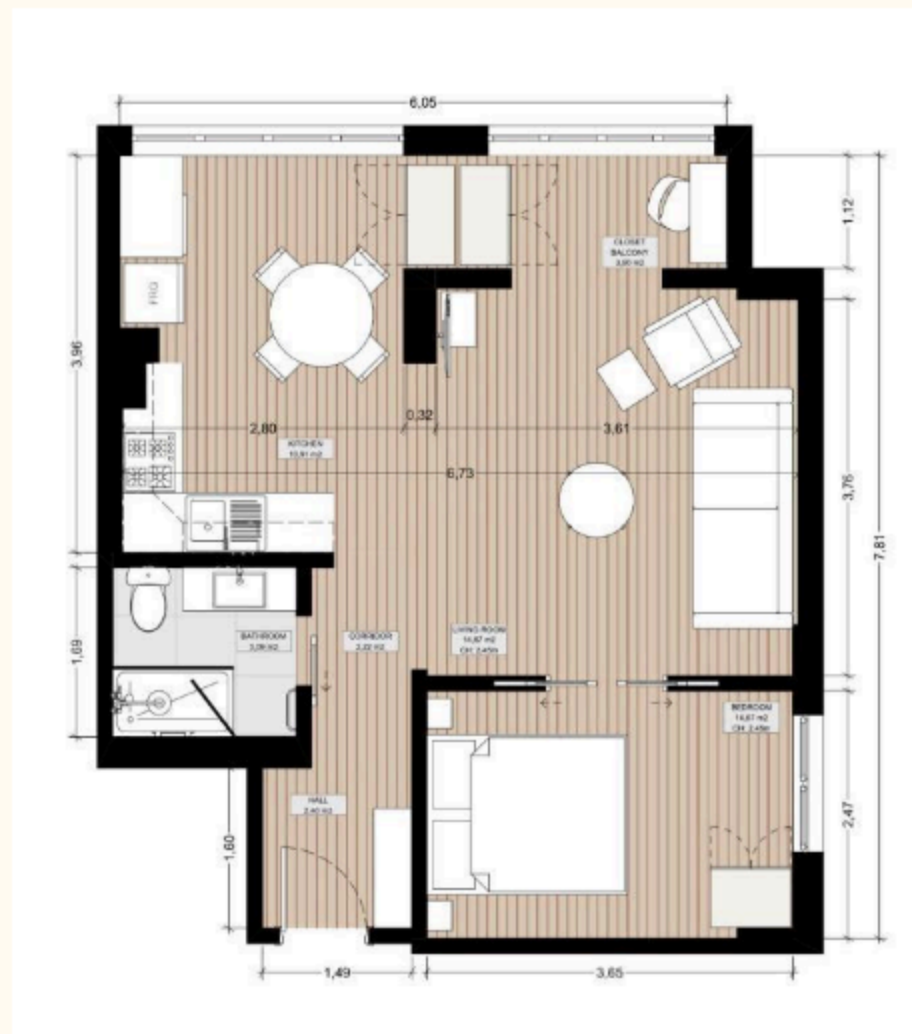
Máy cá nhân: faster-whisper + VieNeu/Supertonic

Render: SpeechRecognition + Piper medium

Mỗi yêu cầu đi qua **bảy bước** trước khi thiết bị thay đổi.



LLM chỉ đề xuất. Backend kiểm tra xong, bộ mô phỏng mới thực hiện.



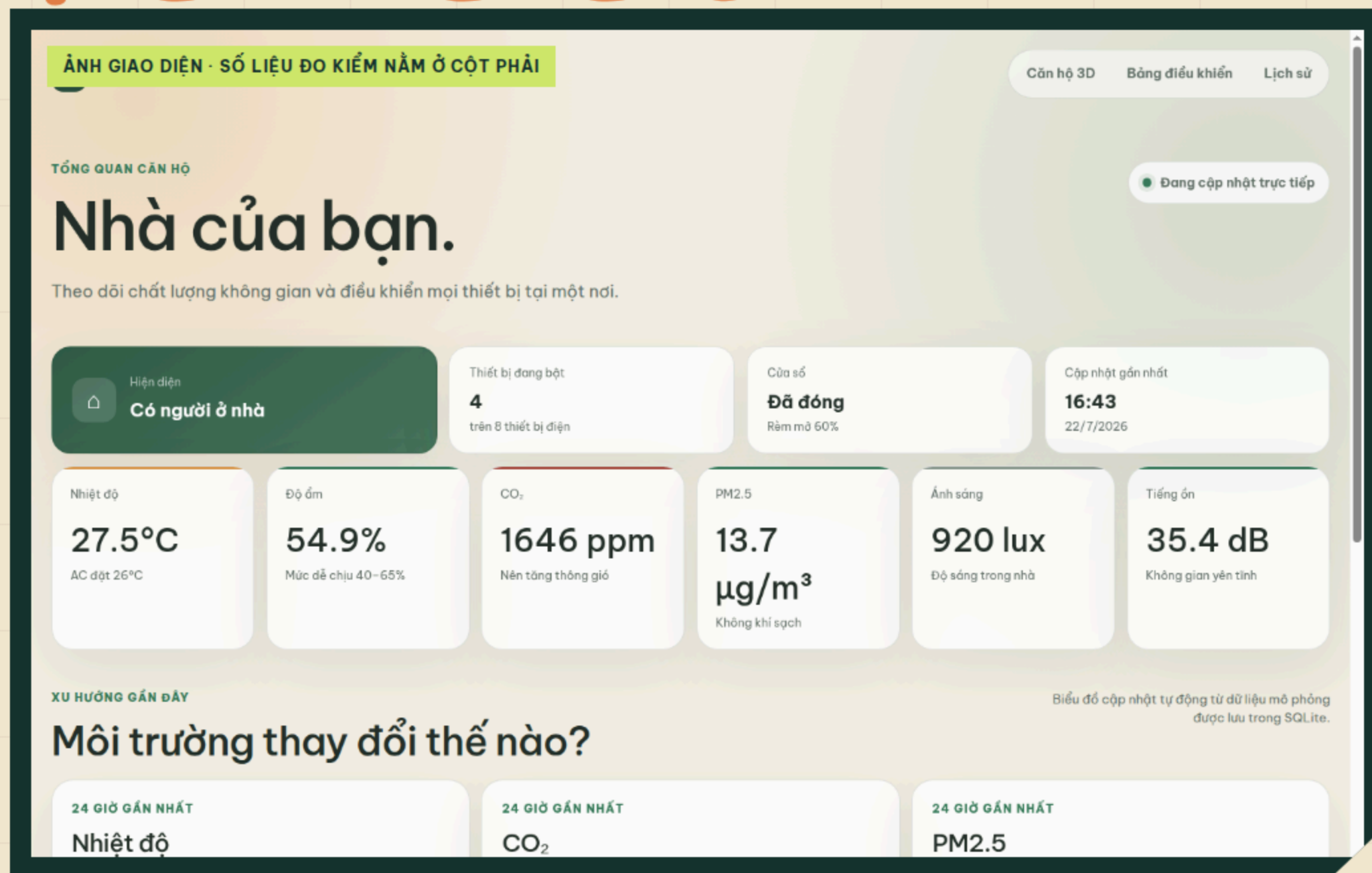
Kịch bản: làm việc, ngủ, nóng, ngọt ngào, ô nhiễm, vắng nhà...

Nhiệt độ, độ ẩm, CO₂, PM2.5, độ rọi và tiếng ồn.

Cùng seed và chuỗi thao tác sẽ cho cùng kết quả.

AC, quạt, đèn, máy lọc không khí, rèm, cửa sổ, máy tính và nguồn điện.

Dữ liệu 24 giờ thay đổi theo từng hoạt động trong ngày.



495–1.036 ppm

CO₂ dao động trong khoảng này; trung vị là 842 ppm.

13,5%

Tỷ lệ mẫu có CO₂ từ 1.000 ppm trở lên, tập trung ở cuối các khoảng làm việc hoặc ngủ.

1.440 mẫu

Mỗi phút ghi một mẫu, bao quát các khoảng làm việc, thư giãn, ngủ, vắng nhà và thông gió.

Hệ thống chỉ tự học sở thích sau khi có đủ bằng chứng.

Người dùng nói rõ

“Khi làm việc, tôi thích đèn vàng 50%.”

Yêu cầu tạm thời

Có thời điểm hết hạn và không trở thành mặc định lâu dài.

Chỉnh lại kết quả

Người dùng sửa đúng thông số mà trợ lý vừa thay đổi.

3×

CÙNG NGỮ CẢNH, CÙNG GIÁ TRỊ

Bằng chứng trong SQLite

Mỗi bằng chứng lưu lại ngữ cảnh, ý định, giá trị, nguồn, độ tin cậy và hạn dùng.

Tìm theo ngữ cảnh

Hệ thống ưu tiên sở thích được nói trực tiếp, tiếp đến sở thích tạm thời, rồi mới dùng sở thích đã học.

Kiểm tra trước khi áp dụng

Sở thích được tìm thấy vẫn phải qua bước kiểm tra lệnh.

Một lần chỉnh chưa làm thay đổi cách xử lý yêu cầu tiếp theo.

Máy cá nhân xử lý giọng nói cục bộ; Render dùng trình duyệt và Piper.

Chạy trên máy cá nhân

MediaRecorder



faster-whisper small



VieNeu / Supertonic

Âm thanh được xử lý ngay trên máy, với các mô hình chạy ở chế độ CPU int8. Nếu microphone lỗi, người dùng vẫn có thể nhập văn bản.

Triển khai trên Render

SpeechRecognition vi-VN



Transcript



Piper medium WAV

Trình duyệt nhận dạng tiếng Việt; backend dùng Piper để tạo giọng đọc ổn định, không phụ thuộc giọng có sẵn trên hệ điều hành.

Dùng chung: chuẩn hóa tiếng Việt · từ điển thuật ngữ · nhập văn bản dự phòng
`/api/tts`

Điểm mạnh lớn nhất của hệ thống là **kết quả có thể được kiểm tra và giải thích.**

Điểm mạnh

- 1 Bộ mô phỏng và dữ liệu có thể tái lập bằng seed.
- 2 Hệ thống lưu nhật ký xử lý, sở thích đã dùng và lịch sử hành động cho từng yêu cầu.
- 3 Vai trò của LLM, backend và lớp thiết bị được tách rõ.

Giới hạn

- 1 Chưa kết nối cảm biến, MQTT hoặc thiết bị vật lý.
- 2 Trợ lý vẫn phụ thuộc vào mô hình bên ngoài và kết nối mạng.
- 3 Bản Render Free chưa có lưu trữ bền vững; thử nghiệm giọng nói còn ít.

● KẾT LUẬN

Điểm cốt lõi là **phân chia trách nhiệm**, không giao toàn quyền cho LLM.

LLM diễn giải yêu cầu, backend kiểm tra lệnh, còn bộ mô phỏng thực hiện và ghi lại kết quả. Cách phân vai này giúp hệ thống dễ kiểm thử hôm nay và sẵn sàng kết nối thiết bị thật sau này.

Hỏi đáp